PES University

Mini Project – Version Control System (PES-VCS)

Name: Ramitha MV | SRN: PES1UG24AM392 | Section: G

2. Objective

To build a local version control system that tracks file changes, stores snapshots efficiently using content-addressable storage, and supports commit history similar to Git.

Git-like System

A Git-like system is a version control system that stores the state of a project as complete snapshots rather than storing differences between files. Each time a commit is made, the entire project structure is saved efficiently. This approach ensures reliability and simplifies tracking changes over time.

Object Types

The system uses three main types of objects. A Blob represents the contents of a file without any metadata. A Tree represents a directory and contains references to blobs and other trees, forming a hierarchical structure. A Commit represents a snapshot of the project and contains metadata such as the author, timestamp, and a reference to the tree.

Key Concepts

The system uses SHA-256 hashing to uniquely identify and store objects based on their content, ensuring data integrity and avoiding duplication. The Index (staging area) keeps track of files prepared for the next commit. The HEAD pointer refers to the current branch, and branches store references to the latest commit, allowing version history to be maintained.

Phase 1: Object Storage

Description

- Implemented object_write and object_read
- Used SHA-256 hashing
- Stored objects in .pes/objects

1.A

```
ramitha-mv@ramitha-mv-VirtualBox:~/PES1UG24AM392-pes-vcs$ nano object.c
ramitha-mv@ramitha-mv-VirtualBox:~/PES1UG24AM392-pes-vcs$ make test_objects
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
ramitha-mv@ramitha-mv-VirtualBox:~/PES1UG24AM392-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc1
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
ramitha-mv@ramitha-mv-VirtualBox:~/PES1UG24AM392-pes-vcs$
```

1.B

```
PASS: integrity check

All Phase 1 tests passed.
ramitha-mv@ramitha-mv-VirtualBox:~/PES1UG24AM392-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
ramitha-mv@ramitha-mv-VirtualBox:~/PES1UG24AM392-pes-vcs$
```

Phase 2: Tree Objects

Description

- Built tree from index
- Serialized directory structure

2.A

```
ramitha-mv@ramitha-mv-VirtualBox:~/PES1UG24AM392 p
com -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
/usr/bin/ld: tree.o: in function 'tree_from_index':
tree.c:(.text+0x3cd): undefined reference to 'index_load'
collect: error: ld returned 1 exit status
make: *** [Makefile:22: test_tree] Error 1
make: *** [Makefile:22: test_tree] Error 1

ramitha-mv@ramitha-mv-VirtualBox:~/PES1UG24AM392 s
/test -/test_tree
```

2.B

```
ramitha-mv@ramitha-mv-VirtualBox:~/PES1UG24AM392 s
find ./pes/objects -type f
.pes/HEAD
.pes/objects/25/eff1fa07ea6852df800dc80756ee6b7ae34b37afedb9b41aef476
.pes/objects/d5/8213f5dbe0629b5c7fa28e5c7d4213ea09227ed0211bbe9db964bafb2
.pes/objects/2a/594d39222787fba8eb7287418aec99c8fc2cedaf5aaf2650ed471e566f
.pes/objects/2a/594d39222787fba8eb7287418aec99c8fc2cedaf5aaaf2650eddfle
```

Phase 3: Index (Staging Area)

Description

- Implemented index_load, index_save, index_add
- Tracks staged files

3.A

```
ramitha-mv@ubuntu:~/PES1UG24AM392$ ./pes status
Staged changes:
  staged:    file1.txt
  staged:    file2.txt
Unstaged changes:
  (nothing to show)
Untracked files:
```

3.B

```
ramitha-mv@ubuntu:~/PES1UG24AM392$ cat .pes/index
100644 2cf8d839dee29543b34a87727421fdec7e3f33a18d3376990
b9ebb4 1776319325 6 file1.txt
e00c50e16a2df38f8d6f809e181ad028da6e66719f35f9f7e65d6636 file2.txt
1776319336 6 file2.txt
```

Phase 4: Commit System

Description

- Implemented commit_create
- Maintains commit history

4.A

```
ramitha-mv@ramitha-mv-VirtualBox:~/PES1UG24AM392-pes-vcs$ /pes log
commit ed8f2eed58b4f2eca07452daf93317123bea203483d0d42E1779X
Author: Your Name <AM392>
Date: 1776320938

  Add farewell

commit c4d062aac2e0e184334bbb0827e74c9a81857c221df2ea2ea09feo715464
Author: Your Name <AM392>
Date: 1776320933
```

4.B

```
ramitha-mv@ramitha-mv-VirtualBox:~/PES1UG24AM392-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/objects/05/6c77d7777dobe85478b78888d1b5ca 77d69754e6680c2C65ca2bd6oc055f4
.pes/objects/16/2246332c36ef49de9c2e7cd6874d61065415sa928308aa1a152d35f4ba77d5693
.pes/objects/4d/c062a2c2e0184334bbb0827e74c9a81857c221df2ea2ea09feo2d715464
.pes/objects/5d/8f2eed58b4f2eca07452daf93311273bea203483d888eb1886652b888b05bc
.pes/objects/97/52ae795284c860d51cbb568202e545878a930517dc2f45eod2b909b054505
.pes/objects/a1/82aaa1d22f782be01271d13738ea01845dd110d42e17779d28f472f
```

4.C

```
ramitha-mv@ubuntu:~/PES1UG24AM392-pes-vcs$ cat .pes/refs/heads/
cat .pes/HEAD
ed8f2eed58b4f2eca07452daf933171273bea203483d0d42e177792d28f4a272
ref: refs/heads/main
ramitha-mv@ubuntu:~/PES1UG24AM392-pes-vcs$ cat .pes/refs/heads/
cat .pes/HEAD
ed8f2eed58b4f2eca07452daf933171273bea203483d0d42e177799d28f4a272
ref: refs/heads/main
```

Branching and Checkout

Q5.1: Implementing pes checkout <branch>

A branch in Git is represented as a file in .pes/refs/heads/ that stores a commit hash. To implement pes checkout <branch>, the system must first update the .pes/HEAD file so that it points to the selected branch reference (e.g., ref: refs/heads/<branch>). Then, it reads the commit hash stored in that branch file and retrieves the corresponding commit object from the object store. From the commit, the associated tree object is obtained, which represents the snapshot of the project.

The working directory must then be updated to match this tree. This involves removing files that are not present in the target tree and restoring files by reading blob objects from the object store. The index must also be updated to reflect the new state of the working directory.

This operation is complex because it involves safely modifying the working directory while preventing data loss. Handling file overwrites, deletions, directory structures, and potential conflicts with uncommitted changes makes checkout a non-trivial operation.

Q5.2: Detecting Dirty Working Directory Conflicts

A dirty working directory refers to the presence of uncommitted changes in tracked files. To detect this using only the index and the object store, the system compares the current working directory with the index entries.

For each file in the index, the system retrieves its stored metadata such as modification time and file size. Using system calls like stat(), it compares these values with the current file in the working directory. If there is a mismatch, the file is considered modified. If a file exists in the index but not in the working directory, it is considered deleted.

Additionally, the system can recompute the hash of the current file and compare it with the stored hash in the index for a more accurate check. If any differences are found, and the file also differs in the target branch, the checkout operation must be aborted to avoid overwriting uncommitted changes.

Q5.3: Detached HEAD State

A detached HEAD state occurs when the .pes/HEAD file contains a commit hash directly instead of a reference to a branch. In this state, any new commits created will not update any branch pointer and are therefore not associated with any branch.

As a result, these commits can become unreachable if the user switches to another branch, since no reference points to them. However, they are not immediately deleted and can still be recovered.

To recover such commits, the user can create a new branch pointing to the commit hash, effectively reattaching the commit to the repository history. Thus, detached HEAD allows temporary exploration of past commits but requires caution to preserve new work.

Garbage Collection and Space Reclamation

Q6.1: Finding and Deleting Unreachable Objects

To identify and delete unreachable objects, the system performs a traversal starting from all branch references stored in .pes/refs/heads/. Each branch points to a commit, and from each commit, the system recursively visits its parent commits and associated tree objects. From the tree objects, it further visits all referenced blobs and subtrees.

A hash set (or similar efficient data structure) is used to store all reachable object hashes during traversal. After collecting all reachable objects, the system scans the entire object store (.pes/objects/) and deletes any object whose hash is not present in the reachable set.

For a repository with 100,000 commits and 50 branches, the traversal would visit all commits reachable from those branches. Since many commits are shared between branches, the total number of unique commits visited would be close to 100,000, along with their associated tree and blob objects.

Q6.2: Dangers of Concurrent Garbage Collection

Running garbage collection concurrently with a commit operation is dangerous due to potential race conditions. During a commit, new objects such as blobs, trees, and the commit itself are created and written to the object store. However, before the commit is fully completed, these objects may not yet be referenced by any branch.

If garbage collection runs at this moment, it may identify these newly created objects as unreachable and delete them. This results in a corrupted repository because the commit will later reference objects that no longer exist.

Git avoids this issue by using mechanisms such as locking, temporary references, and careful ordering of operations. For example, it ensures that objects are written and references are updated atomically, and garbage collection does not run concurrently with critical write operations. This guarantees that no valid objects are mistakenly deleted.